



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CURSO DE GRADUAÇÃO EM CIÊNCIAS DA COMPUTAÇÃO

Rafael Veronezi Ribeiro

Projeto II

Rafael Veronezi Ribeiro

Projeto II

Identificação de Prefixos e Indexação de Dicionários

Relatório técnico apresentado como requisito parcial para avaliação na disciplina de Estruturas de Dados, do curso de Ciências da Computação da Universidade Federal de Santa Catarina.

Professor: Alexandre Goncalves Silva

Florianópolis, 2026

Resumo

Este relatório descreve o desenvolvimento de uma solução em C++ baseada na estrutura de dados *Trie* (Árvore de Prefixos) para a indexação e recuperação eficiente de palavras em grandes arquivos de dicionários. O sistema foi projetado para solucionar dois problemas principais: a identificação de prefixos em um conjunto de palavras e a indexação física do arquivo, mapeando a posição inicial em bytes e o comprimento da linha correspondente a cada termo definido no dicionário. A implementação foi capaz de processar o arquivo texto caractere por caractere, montar a hierarquia em memória principal e responder a múltiplas consultas de forma otimizada, validando se uma entrada é prefixo de outras palavras ou não.

Palavras-chave: Trie; estrutura de dados; indexação de ficheiros; identificação de prefixos; C++.

Sumário

1	Introdução	3
2	Desenvolvimento	3
2.1	Estrutura do Nó da Trie	3
2.2	Indexação e Varredura do Arquivo	4
2.3	Algoritmos de Busca	4
3	Considerações Finais	5
3.1	Dificuldades e Ambiguidade Encontradas	5

1 Introdução

O armazenamento e a recuperação rápida de informações em grandes volumes de dados são desafios clássicos na ciência da computação. Quando lidamos com dicionários de palavras, realizar buscas lineares em arquivos mantidos na memória secundária torna-se um processo custoso e ineficiente.

O objetivo deste projeto foi construir uma estrutura hierárquica denominada *Trie* (do inglês “*retrieval*”) em memória principal. A árvore digital permite que o tempo de busca por uma palavra seja proporcional ao comprimento da própria palavra, e não à quantidade de itens no dicionário. O trabalho exigiu a leitura de arquivos de dicionário específicos, onde as palavras-alvo são demarcadas por colchetes [], e a posterior consulta de termos variados. Além de confirmar a existência de prefixos, a solução precisou indexar fisicamente o arquivo, guardando a posição em bytes do início da linha e seu respectivo tamanho. Este relatório detalha as estruturas escolhidas, os algoritmos de varredura desenvolvidos e discute os resultados e desafios encontrados.

2 Desenvolvimento

Para atender aos requisitos de indexação e busca, o projeto foi subdividido em três etapas lógicas: a definição da estrutura de dados, o algoritmo de leitura e indexação, e o algoritmo de busca e contagem.

2.1 Estrutura do Nó da Trie

A base do projeto é a `struct NoTrie`. Em vez de armazenar *strings* completas em cada nó, a *Trie* divide as palavras em caracteres. Cada nó armazena seu próprio caractere (letra) e um vetor de ponteiros para até 26 filhos, representando o alfabeto minúsculo sem caracteres especiais.

Adicionalmente, os nós que representam o final de uma palavra válida no dicionário carregam duas variáveis cruciais de indexação: `posicao` e `comprimento`. Se o comprimento for maior que zero, o nó não apenas compõe um prefixo, mas também sinaliza o término de um termo indexado do dicionário.

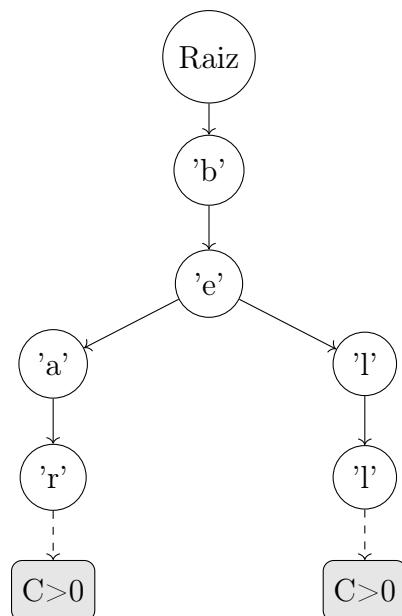


Figura 1: Representação visual da Trie acomodando as palavras 'bear' e 'bell'. Nós com marcação $C>0$ indicam o término da palavra e o armazenamento da posição e comprimento da linha.

2.2 Indexação e Varredura do Arquivo

Um dos pontos mais críticos do desenvolvimento foi o mapeamento dos bytes. A leitura em C++ foi feita utilizando `arquivo.get(c)` dentro de um laço `while`, lendo o arquivo caractere por caractere para garantir controle total sobre a posição real na memória secundária.

Sempre que o caractere de abertura de colchetes `[` é detectado, a variável `byteAtual` é salva como a posição inicial da palavra. O algoritmo então extrai a palavra até encontrar o fechamento `]` e continua avançando e incrementando o `byteAtual` até encontrar uma quebra de linha `\n`, momento em que o tamanho total da linha é calculado e a função `inserePalavra` é chamada para popular a *Trie*.

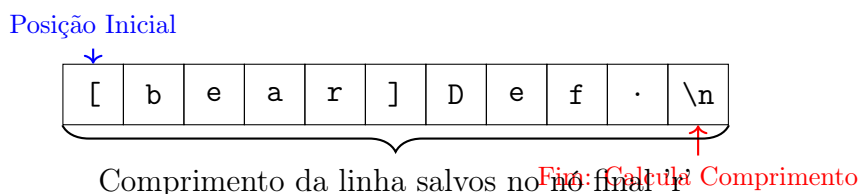


Figura 2: Processo de indexação. O ponteiro de leitura localiza o caractere de abertura '[' para armazenar a posição, e percorre até a quebra de linha para calcular o comprimento total.

2.3 Algoritmos de Busca

A busca é processada navegando no vetor de filhos correspondente ao índice da letra ($\text{indice} = c - 'a'$). Se um ponteiro for nulo durante a travessia de uma *string* recebida, a palavra solicitada não existe na árvore e o programa sinaliza "[palavra] is not prefix".

Caso a travessia seja bem-sucedida, é necessário contar de quantas palavras no dicionário aquele nó atua como prefixo. Para isso, foi desenvolvida a função recursiva `contaPrefixos`.

```
1 int contaPrefixos(NoTrie *no) {
2     if (no == nullptr) return 0;
3
4     int total = 0;
5     if (no->comprimento > 0) total++; // E o fim de uma palavra valida
6
7     for (int i = 0; i < 26; i++) {
8         if (no->filhos[i] != nullptr) {
9             total += contaPrefixos(no->filhos[i]); // Varredura profunda
10        }
11    }
12    return total;
13 }
```

Listing 1: Trecho de código demonstrando a contagem de prefixos de forma recursiva.

Esta função percorre todos os ramos abaixo do nó final da palavra consultada, somando 1 ao total sempre que encontra um comprimento maior que 0, indicando que ali termina uma palavra indexada.

3 Considerações Finais

O algoritmo atendeu a todos os critérios estabelecidos. Durante a execução simulada com o arquivo de teste (onde são inseridas várias palavras, finalizadas pelo dígito "0"), o programa conseguiu imprimir corretamente não apenas a quantidade de ramificações que compartilham aquele prefixo, mas também a sua localização em coordenadas (Posição, Comprimento).

3.1 Dificuldades e Ambiguidade Encontradas

Embora a implementação tenha sido fluida de modo geral, alguns pontos merecem discussão por apresentarem desafios técnicos e espaço para melhoria:

Controle Preciso de Quebras de Linha: Diferentes sistemas operacionais tratam quebras de linha de forma distinta (ex: `\n` em sistemas baseados em Unix contra `\r\n` no Windows). A leitura byte a byte usando `get(c)` exige muito rigor lógico. Se o arquivo contiver duplos espaços ou formatações invisíveis inconsistentes, a contagem de `byteAtual` pode desalinhar da leitura padrão do sistema.

Gestão de Memória: A alocação dinâmica em estruturas *Trie* consome uma quantidade significativa de memória, pois cada caractere cria um novo nó que, por sua vez, reserva espaço para 26 ponteiros de possíveis filhos. Além disso, nota-se que, devido à natureza do ciclo de vida do programa, não foi implementada uma rotina recursiva de “destruição” da *Trie* (como uma função para realizar o `delete` dos nós e liberar a memória no final da `main`). Para execuções pontuais em laboratório isso não afeta os testes do VPL, mas em um cenário de produção em larga escala resultaria em vazamento de memória (*memory leak*).

Alternativas de Implementação: Uma alternativa pertinente para lidar com o alto consumo de memória da *Trie* tradicional seria a implementação de uma *Patricia Trie* (ou *Radix Tree*). Essa estrutura comprime caminhos lineares onde há apenas um filho, agrupando *strings* em um único nó, o que economizaria um espaço valioso considerando que grande parte das palavras em dicionários naturais possuem terminações longas e singulares.

Referências

- [1] Sedgewick, Robert. *Algorithms in C++, Parts 1-4: Fundamentals, Data Structure, Sorting, Searching*. 3^a edição. Addison-Wesley Professional, 1998.
- [2] Morrison, Donald R. “PATRICIA—Practical Algorithm To Retrieve Information Coded in Alphanumeric”. *Journal of the ACM (JACM)*, vol. 15, n^o. 4, pp. 514-534, 1968.
- [3] Silva, Alexandre Gonçalves. *Estruturas de Dados: Projeto II - Identificação de Prefixos e Indexação de Dicionários*. Material de apoio e especificação do projeto da disciplina INE5408. Universidade Federal de Santa Catarina (UFSC) - Moodle Presencial, 2026. Disponível em: <https://presencial.moodle.ufsc.br/mod/vpl/view.php?id=1290013>.